

Plano: gerar a app "BioByte Sentinela" (ICSAC) 100% pela UI, capturar cada tela em PNG e montar o roteiro do vídeo

Context

O user quer um exemplo pequeno, útil e completo para a plateia (empresa **BioByte**, controle de infecção hospitalar / IRAS): uma app de **vigilância de Infecção de Corrente Sanguínea Associada a Cateter (ICSAC)**, gerada **inteira pela UI do LangNet**, passando por **todas** as etapas do pipeline, **criando e usando 2 tools MCP** (uma delas é um algoritmo do PDF virando serviço externo), e **rodando o app no final**. Cada tela deve ser **capturada em PNG** e cada ação **registrada como texto de narração** — o user vai gerar o vídeo depois, colando os PNGs + a narração. Decisões do user: **ICSAC** · **2 tools MCP** (`consultar_microbiologia` + `escore_risco_cox`) · **gerar + RODAR o app E2E**. LLM: **qwen local** (LM Studio), nunca DeepSeek cloud. Regra dura: **tudo pela UI** (gerar/refinar/aprovar); só a criação dos servidores MCP é externa (registro/atribuição é pela UI).

App a gerar (design mínimo)

- **Domínio**: Saúde · **Framework**: CrewAI · **Protocolo**: OKF.
- **Entidades (~5)**: `paciente` (uti, comorbidades, neutropenia) · `dispositivo` (cateter_central, data_insercao, dias_uso, nutricao_parenteral) · `cultura` (patogeno, resistencia, fonte) — vem via MCP · `avaliacao_iras` (escore_risco, classificacao_caso ENUM, conduta, rra_estimada, status) · `laudo_ccih`.
- **3 agentes (um por área do PDF)**:
 - `vigilancia_agent` — **Prognóstico**: escore de risco de ICSAC (Cox) → prioriza. Usa MCP `escore_risco_cox`.
 - `diagnostico_agent` — **Diagnóstico**: importa microbiologia (MCP `consultar_microbiologia`) + aplica critério NHSN + classifica (Regressão Logística/Florestas) → IRAS confirmada? multirresistente?
 - `conduta_agent` — **Tratamento**: recomenda bundle/conduta + estima Efeito Médio do Tratamento/RRA → laudo CCIH.
- **~6 requisitos funcionais** (um por passo): FR-01 cadastro paciente+dispositivo · FR-02 escore de risco (Cox, MCP) · FR-03 importar microbiologia (MCP) · FR-04 classificar caso NHSN + multirresistência · FR-05 conduta + redução de risco · FR-06 laudo/notificação CCIH. NFRs: LGPD, auditabilidade/rastreabilidade, latência.
- **UC central**: UC-001 "Avaliar paciente sentinela" (cadastro → risco → microbiologia+classificação → conduta+laudo).

As 2 tools MCP (criação → registro → atribuição, reusando o E2E já provado)

1. **Criar/rodar 2 servidores MCP** (FastMCP SSE, como no E2E de hoje), em `docs/apresentacao/demo-biobyte/mcp-servers/` :
2. `microbiologia_server.py` (:9110) → tool `consultar_microbiologia(paciente_id)` = hemocultura+antibiograma (LIS externo, base estática+fallback).
3. `escore_server.py` (:9111) → tool `escore_risco_cox(dias_cateter, uti, nutricao_parenteral, neutropenia, idade)` = escore de risco (fórmula tipo Cox) → o "algoritmo do PDF como tool MCP".
4. **Registrar pela UI em MCP → Configuração Global** (`/mcp/config` , `McpServersManager`): "Registrar Servidor MCP" (SSE, url) → "Testar Conexão" (descobre a tool). Capturar.
5. **Atribuir pela UI em MCP do Projeto** (`/project/:id/mcp` , `McpProjectManager`): habilitar os 2 servidores → atribuir `consultar_microbiologia` → `diagnostico_agent` e `escore_risco_cox` → `vigilancia_agent` (ou usar "Sugerir Atribuições"). Capturar. (Grava em `mcp_agent_tools` ; consumido na Geração de Código → emite `ws-server/mcp_tools.py` .)
6. Fazer **depois** do estágio Agentes & Tarefas / YAML (os agentes precisam existir) e **antes** da Geração de Código.

Harness de captura (reusar o de hoje) + log de narração

- Estender `tools/langnet_demo_capture.js` : Playwright headless, `addInitScript` injeta user (Admin Master) + token **antes** do load, `ctx.route('**/api/**')` encaminha p/ :8000 com CORS (contorna o dev-server). Assim a UI carrega dados reais e o contexto de projeto (menu do pipeline) aparece. `deviceScaleFactor:1.5`.
- **Todas as PNGs** vão para `docs/apresentacao/demo-biobyte/shots/` com nome por etapa (ex.: `03_data_model_stage.png` , `03_data_model_schema.png` , `03_data_model_doc.png`).
- **Registrar a narração**: manter `docs/apresentacao/demo-biobyte/narracao_log.md` — a cada tela capturada, uma linha "o que estou fazendo / o que a tela mostra" (fonte crua da narração do vídeo).
- Token: eu minto (`/tmp/langnet_token.txt` , 7 dias); eu subo o backend :8000 (sem pkll do recém-subido).

Runbook — passo a passo, com o documento gerado, as PNGs e a narração (NÃO pular nenhuma etapa)

#	Etapa (rota)	Ação na UI	Documento/artefato gerado	PNGs a capturar	Narração (registrar)
0	Projetos (/ projects)	"Criar Novo Projeto": nome "BioByte Sentinela", domínio Saúde , Framework CrewAI , Protocolo OKF	projeto criado (draft)	00_projetos.png , 00_criar_projeto_modal.png (com Framework+Protocolo)	"Criamos o projeto de vigilância d CrewAI, protocolo OKF."
0b	Menu lateral	Abrir projeto → sidebar do pipeline	—	00b_sidebar.png	"O menu lateral revela todas as e
1	Documentos (/ documents)	" + Upload" do brief-semente (ICSAC + 3 algoritmos + 2 integrações MCP) → "Instruções para Análise" → "Iniciar Análise"	Documento de Requisitos (FR/NFR)	01_doc_stage.png , 01_req_fr.png , 01_req_nfr.png	"Subimos o brief e geramos o doc FR-016... (risco), microbiologia, c
2	Especificação (/spec)	" Gerar" → " Revisar"/"Refinar" → Aprovar → "👁 Visualizar"	Especificação (casos de uso, fluxos, wireframe)	02_spec_stage.png , 02_spec_uc.png , 02_spec_fluxos.png , 02_spec_matriz.png	"A especificação detalha o UC-00 sentinela) e a matriz FR → UC."
3	Modelo de Dados (/data-model)	DBMS PostgreSQL → " Gerar" → abas Entidades/SQL/models.py/ Alembic → Aprovar	entities.json, schema.sql, models.py, alembic, yaml	03_dm_stage.png , 03_dm_schema.png , 03_dm_models.png	"Modelo de dados: paciente, disp avaliacao_iras."
4	Interface & Protótipo (/ui-spec)	" Gerar" → aba Telas + mockup → " Coerência" → Aprovar	ui_spec.json, telas/ mockups, coherence_report	04_ui_stage.png , 04_ui_tela_painel.png (painel sentinela), 04_ui_coerencia.png	"Protótipo: painel de priorização + paciente."
5	Agentes & Tarefas (/ agent-task)	Nível de detalhe + framework CrewAI → " Gerar" → "👁 Visualizar"	ATS (agentes + tarefas + campo Tools)	05_at_stage.png , 05_at_agentes.png , 05_at_tools.png (campo Tools por agente)	"Definimos 3 agentes (vigilância, as tarefas — com o campo Tools
6	YAML de Agentes e Tarefas (/ yaml-generation)	aba Agents YAML / Tasks YAML → " Gerar" → "👁 Visualizar"	agents.yaml, tasks.yaml (traceability nas tasks)	06_yaml_agents.png , 06_yaml_tasks.png	"Os agentes e tarefas viram YAML rastreabilidade impressa."
6.5	MCP Global (/ mcp/config)	Registrar microbiologia (:9110) e escore_risco_cox (:9111) → "Testar" (descobre tools)	servidores em mcp_servers	65_mcp_global_registro.png , 65_mcp_global_tools_descobertas.png	"Registramos 2 servidores MCP e as tools."
6.6	MCP do Projeto (/ project/:id/ mcp)	Habilitar os 2 servidores → atribuir tools aos agentes (ou "Sugerir Atribuições")	mcp_agent_tools	66_mcp_projeto_habilitar.png , 66_mcp_projeto_atribuir.png	"Plugamos: diagnóstico – consulta vigilância – escore_risco_cox (o a tool MCP)."
7	Sequência de Tarefas (/ task-execution-flow)	" Gerar Rede/ Origem" (spec+ATS+tasks.yaml) → " Gerar" → "👁 Visualizar"	task_flow (ordem/ dependências)	07_seq_stage.png , 07_seq_doc.png	"A sequência ordena as tarefas: cadastro → risco → microbiologia →
8	Rede de Petri (/petri-net)	" Gerar Rede" (yaml+sequência) → " Gerar Rede" → (Simular)	Rede de Petri (lugares/ transições)	08_petri_stage.png , 08_petri_sim.png	"A orquestração formalizada com
9	Geração de Código (/ code-generation)	" Gerar" (agents.yaml+tasks.yaml+Petri) → árvore de arquivos + Monaco	projeto completo + ws-server/mcp_tools.py (das atribuições MCP)	09_code_stage.png , 09_code_mcp_tools.png (mcp_tools.py), 09_code_func.png	"Geramos o código — e o mcp_to wrappers das 2 tools MCP."
10	Casos de Teste & Validação (/ test-cases)	Selecionar UC → " Gerar" → abas Grafo CEG / Tabela de Decisão / Casos	CEG + tabela de decisão + casos de teste	10_tests_stage.png , 10_tests_ceg.png , 10_tests_tabela.png	"Os casos de teste derivam do UC causa-efeito."
11	Deploy (/ deploy) — mock	abrir a tela	(mock)	11_deploy_mock.png	"Etapa de Deploy (em desenvolvi de operação."
12	Monitoramento (/ monitoring) — mock	abrir a tela	(mock)	12_monitor_mock.png	"Etapa de Monitoramento (em des pipeline."
13			app rodando + chamada MCP real	13_run_console.png , 13_app_home.png ,	

#	Etapa (rota)	Ação na UI	Documento/artefato gerado	PNGs a capturar	Narração (registrar)
	Rodar o app (Código → ▶ Executar")	Executar o ws-server; abrir o app gerado; disparar o fluxo de um paciente		13_app_avaliacao.png , 13_app_mcp_call.png	"Rodamos o app: o agente de diálogo MCP de microbiologia de verdade deu o escore Cox."

Notas de execução: cada "Gerar" é uma chamada ao **gwen local** (minutos por etapa) — executar por etapa, capturar, aprovar, seguir. Se um estágio sumarizar/perder detalhe, usar "Refinar com o agente" (não "Regenerar") com instruções explícitas (fio condutor: escore de risco, microbiologia, RRA). Portão de rastreabilidade deve ficar verde ao final (`tools/langnet_trace_gate.py` `<project_id>`); capturar `14_gate_verde.png` .

Vídeo (esquema final)

- Gerar `docs/apresentacao/demo-biobyte/script_video_biobyte.{md,pdf}` no MESMO formato do `demo-s61` (via um `demo_build.py` local): por cena → **NARRAÇÃO** (fala exata, siglas por extenso), **PRODUÇÃO** (o que mostrar) e o **PNG** da tela. Ordem = o runbook acima (Projetos → cada etapa → MCP → Código → Testes → App). Consolidar a `narracao_log.md` (registro do que fez) como a fonte da locução. O user pega os PNGs + a narração e gera o vídeo.

Arquivos/saídas (tudo em `docs/apresentacao/demo-biobyte/`)

- `mcp-servers/microbiologia_server.py` , `mcp-servers/escore_server.py`
- `seed/brief_icsac.md` (brief-semente para o estágio Documentos)
- `shots/*.png` (todas as telas do runbook)
- `narracao_log.md` (registro/narração crua)
- `demo_build.py` + `script_video_biobyte.{md,pdf}` (roteiro final)
- `tools/langnet_demo_capture.js` estendido (harness de captura)

Verificação (E2E, tudo pela UI)

1. Projeto "BioByte Sentinela" criado (Saúde/CrewAI/OKF); sidebar do pipeline aparece.
2. Cada etapa gerada e **aprovada** pela UI, com o documento correspondente capturado (Requisitos → ... → Casos de Teste).
3. As 2 tools MCP registradas (handshake OK) e **atribuídas** aos agentes; a Geração de Código emite `ws-server/mcp_tools.py` com os 2 wrappers (grep no arquivo gerado).
4. Portão de rastreabilidade VERDE (FR → UC → código) para o projeto.
5. App **rodando**: um agente chama `consultar_microbiologia` e outro `escore_risco_cox` de verdade (log/console).
6. `script_video_biobyte.pdf` gerado, uma cena por tela do runbook, com narração + PNG.

Escopo / bom senso

Pequeno de propósito (5 tabelas, 3 agentes, 1 UC central, 2 tools) para caber num vídeo curto e num ciclo de geração rápido — mas cobrindo **todas** as etapas + MCP + execução. Correções sempre pela UI (Refinar/Aprovar); nada de editar artefato à mão. Se o LLM local travar numa etapa grande, dividir a instrução (estratégia anti-timeout) e refinar. As etapas Deploy/Monitoramento são mock — capturadas e narradas como "em desenvolvimento", sem aprofundar.